Universidade Municipal de São Caetano do Sul

ENGENHARIA DE SOFTWARE
GABARITO

São Caetano do Sul
2026

Questão 1 - Gabarito: C

Justificativa Geral: A técnica JAD (Joint Application Development) é caracterizada por reunir diversos perfis de stakeholders (usuários, técnicos, gerentes) em workshops estruturados e intensivos, guiados por um facilitador, visando chegar a um consenso e documentar requisitos rapidamente, alinhando expectativas.

Análise dos Distratores:

- **A) Incorreta.** Observação (ou Etnografia) envolve assistir o usuário trabalhando passivamente, não em uma reunião imersiva conjunta.

- **B) Incorreta.** O Brainstorming é focado apenas na ideação livre, não possuindo o rigor do JAD para definição formal e aprovação documentada de requisitos.

- **D) Incorreta.** Questionários são ferramentas assíncronas e individuais aplicadas a grandes públicos, sem a interação de um workshop.

Questão 2 - Gabarito: A

Justificativa Geral: A Asserção I é verdadeira pois Requisitos Não Funcionais (RNFs) devem evitar ambiguidades, sendo estipulados através de métricas precisas. A Asserção II também é verdadeira, pois esses requisitos ditam atributos de qualidade (como performance, segurança, disponibilidade) que são a espinha dorsal da arquitetura do software. A relação de causa (PORQUE) existe: a exigência de quantificar e testar objetivamente os RNFs (I) ocorre justamente pelo fato de que eles afetam estruturalmente a arquitetura e não podem ser consertados facilmente depois, exigindo rigor em sua definição prévia (II).

Análise dos Distratores:

- **B) Incorreta.** A asserção II complementa e justifica perfeitamente o rigor demandado na asserção I.

- **C e D) Incorretas.** Nenhuma das proposições é falsa. Ambas trazem conceitos fundamentais consolidados da Engenharia de Requisitos e Arquitetura de Software.

Questão 3 - Gabarito: B

Justificativa geral: A alternativa B expressa exatamente a boa prática descrita na Apostila para cenários de gestão de mudanças e rastreabilidade. A Matriz de Rastreabilidade permite acompanhar a vida do requisito nas duas direções: *backward* (conectando o requisito à sua fonte/origem ou norma legal) e *forward* (conectando o requisito aos componentes de software, código e casos de teste). Ferramentas corporativas como IBM DOORS e Jira automatizam a análise de impacto, prevenindo inconsistências em sistemas regulados.

Análise dos Distratores:

A) Incorreta: O modelo cascata rígido tenta congelar requisitos, o que falha em ambientes com mudanças regulatórias inevitáveis.

C) Incorreta: Eliminar a documentação de requisitos em sistemas regulados e auditáveis fere gravemente as diretrizes da Engenharia de Requisitos e as exigências da LGPD/órgãos reguladores.

D) Incorreta: A prototipagem é excelente para validação de interface, mas não substitui a rastreabilidade formal de testes e código.

Questão 4 - Gabarito: B

Justificativa Geral: A alternativa B diagnostica com precisão a violação do Princípio da Responsabilidade Única (SRP) do SOLID. Uma classe com múltiplas razões para mudar (cálculo de frete, regras de desconto, formatação em PDF e persistência SQL) é uma *God Class* (Classe Deus). A refatoração correta exige a separação dessas responsabilidades em classes altamente coesas (CalculadoraDeFrete, GeradorDeReciboPdf e PedidoRepository).

Análise dos Distratores:

A) Incorreta: O LSP trata da capacidade de uma subclasse substituir sua superclasse sem quebrar a aplicação. Lançar exceções em subclasses viola o LSP.

C) Incorreta: O ISP defende a criação de interfaces pequenas e específicas em vez de interfaces "gordas" e unificadas.

D) Incorreta: Modificar o código fonte existente a cada nova regra viola o Princípio do Aberto/Fechado (OCP), que orienta estender funcionalidades via abstrações/polimorfismo sem alterar a classe base.

Questão 5 - Gabarito: B

Justificativa Geral: A alternativa B traz a análise exata de *trade-offs* descrita na Primeira Lei da Arquitetura de Software. A adoção de microsserviços oferece escalabilidade isolada (permitindo alocar recursos computacionais especificamente no módulo de *checkout*) e tolerância a falhas (impedindo que a lentidão em relatórios derrube o banco inteiro). Todavia, o custo assumido envolve o aumento exponencial na complexidade de orquestração, redes, latência HTTP/mensageria e governança de dados.

Análise dos Distratores:

A) Incorreta: Inverte a realidade: microsserviços *aumentam* a complexidade de rede e *ganham* independência de implantação.

C) Incorreta: Microsserviços utilizam bancos de dados independentes e descentralizados, o que dificulta transações ACID distribuídas.

D) Incorreta: A arquitetura de microsserviços depende obrigatoriamente de APIs e *Gateways* para rotear requisições entre os clientes e os serviços.

Questão 6 - Gabarito: B

Justificativa geral: A alternativa B demonstra a correta interpretação integrada das métricas de software. Uma complexidade ciclomática média alta (24) indica que o código possui muitos caminhos e ramificações condicionais (if/else/switch). Quando combinada com uma baixa cobertura de testes unitários de caixa-branca (42%), significa que a maioria dos caminhos internos do código não é testada, gerando a alta densidade de defeitos (4.8 defeitos/KLOC) observada. A solução exige refatoração para simplificar o código e ampliação de testes unitários.

Análise dos Distratores:

A) Incorreta: A alta cobertura de testes de caixa-preta (98%) testa apenas as entradas e saídas de requisitos de alto nível, sendo incapaz de detectar falhas em fluxos de exceção internos de um código complexo.

C) Incorreta: Aumentar a complexidade ciclomática piora a manutenibilidade e aumenta a quantidade de bugs.

D) Incorreta: Lançar o software sem tratar a causa dos defeitos agrava os custos de manutenção em produção.

Questão 7 - Gabarito: C

Justificativa Geral: No framework Scrum, os papéis são muito bem definidos. O Product Owner (PO) é o responsável por definir a visão do produto, maximizar o valor entregue e gerenciar/priorizar o Product Backlog (perfil das atividades de João). Já o Scrum Master (SM) atua como um líder servidor, facilitando os processos, orientando a equipe sobre as práticas ágeis e atuando ativamente na remoção de impedimentos (perfil das atividades de Maria).

Análise dos Distratores:

A) Incorreta. Os papéis estão invertidos na descrição da alternativa.

B) Incorreta. "Líder Técnico" e "Gerente de Projetos (PMBOK)" representam terminologias e papéis de outras metodologias e estruturas organizacionais, não pertencendo aos papéis oficiais do framework Scrum.

D) Incorreta. "Gerente de Portfólio" não é um papel ágil aplicável ao contexto descrito; Maria não foi descrita realizando atividades puras de codificação (Desenvolvedora)

Questão 8 - Gabarito: C

Justificativa Geral: Diferentes contextos de projeto exigem diferentes abordagens. O Sistema A, com requisitos estáveis e previsíveis, beneficia-se de um modelo tradicional (sequencial/preditivo). O Sistema B, lidando com alta incerteza e necessidade de feedback constante, necessita obrigatoriamente de uma abordagem ágil/iterativa.

Análise dos Distratores:

- **A) Incorreta.** Embora a agilidade seja popular, metodologias preditivas ainda são altamente recomendadas para projetos estáveis e previsíveis, não sendo obrigatório o uso ágil exclusivo.

- **B) Incorreta.** Utilizar um fluxo rígido para requisitos incertos é contraproducente, pois o custo da mudança (quando os requisitos se revelarem) será altíssimo.

- **D) Incorreta.** Exatamente o oposto: se os requisitos não estão definidos, o modelo Cascata (que exige requisitos fechados desde o início) é o mais inadequado.

Questão 9 - Gabarito: B

Justificativa Geral: A afirmativa I está correta pois o modelo evolucionário foca na adaptação por meio de ciclos contínuos baseados no feedback. A afirmativa II está correta, já que a criação de versões simplificadas (interfaces) para validação inicial de requisitos com o usuário é a definição da técnica de Prototipação. A afirmativa III está incorreta.

Análise dos Distratores:

III) Falsa. O processo descrito (iterativo, focado em validação constante, criação de protótipos e avanço gradual) é diametralmente oposto ao Modelo Cascata, que é rígido, sequencial e exige o levantamento total dos requisitos antes de qualquer implementação.

As alternativas A, C e D estão incorretas pois não contemplam a combinação exata das afirmações verdadeiras (I e II).

Questão 10 - Gabarito: C

Justificativa Geral: O modelo Espiral (I) prioriza a análise contínua de riscos, ideal para sistemas críticos como o Cenário X. A Prototipação Evolucionária (III) permite feedback rápido, ideal para validar mercado como exigido no Cenário Y. O RUP na fase de Elaboração (IV) foca na estabilização arquitetural e mitigação de riscos, adequado para o Cenário X.

Análise dos Distratores:

II) Falsa. O modelo Cascata não adota contato precoce com o cliente nem entregas iterativas; ele codifica e testa tudo de uma vez. Para o Cenário Y (startup, extrema incerteza), essa abordagem tem alto risco de gerar um produto sem valor de mercado, sendo inadequada.

As alternativas A, B e D estão incorretas pois incluem a afirmação II (falsa) ou excluem alguma das afirmações corretas (I, III ou IV).

Questão 11 - Gabarito: A

Justificativa Geral: Ambas as asserções são verdadeiras e a II justifica corretamente a I. Na notação UML de casos de uso, o <<include>> é usado quando um comportamento é sempre executado como parte do fluxo do caso de uso base — a seta é desenhada do caso de uso base ("Finalizar Compra") para o caso de uso incluído ("Autenticar Usuário"), pois é o primeiro que depende do segundo para se completar. Já o <<extend>> modela um comportamento opcional, inserido apenas sob certas condições (pontos de extensão) — nesse caso, a seta é desenhada em sentido contrário, do caso de uso que estende ("Aplicar Cupom de Desconto") para o caso de uso base ("Finalizar Compra"), pois é o caso de uso base que é estendido, e não o inverso. Esse ponto — a direção da seta em cada estereótipo — é um dos erros mais comuns em provas de Engenharia de Software, já que intuitivamente muitos candidatos desenham as duas setas no mesmo sentido.

Análise dos Distratores:

B) Incorreta.
As afirmações I e II são verdadeiras, mas a II justifica a I, sim. Portanto, não é possível dizer que a II não explica a I.

C) Incorreta.
A afirmação I é verdadeira, porém a II também é verdadeira. Ela não é falsa, pois descreve corretamente a finalidade de <<include>> e <<extend>>.

D) Incorreta.
A afirmação II é verdadeira, mas a I também é verdadeira. Os relacionamentos e a direção das setas estão corretos.

Questão 12 – Gabarito C

Justificativa Geral :

I (Verdadeira): A autenticação é um pré-requisito obrigatório e incondicional para a execução da transferência. Portanto, a relação é de inclusão obrigatória (<<include>>).

• **II (Verdadeira):** A validação por biometria facial só ocorre condicionalmente quando o valor excede R$ 5.000,00. Trata-se do clássico comportamento opcional do <<extend>>, partindo do caso de uso estendido para o caso de uso base.

• **III (Falsa):** O registro de log ocorre apenas em situações excepcionais de falha/erro. Como não ocorre em todas as execuções normais do caso de uso base, a relação correta seria <<extend>> (com condição de exceção) ou um tratamento interno de exceção, jamais <<include>> (que exigiria execução incondicional).

• **IV (Verdadeira):** Na UML, o relacionamento de generalização entre atores permite que o ator especializado herde todas as associações com casos de uso do ator pai.
Conclusão: Afirmativas I, II e IV estão corretas.

Questão 13 – Gabarito C

Justificativa Geral : A asserção I é verdadeira: o diagrama de implantação é, de fato, o diagrama estrutural da UML voltado a representar a topologia física do sistema em produção nós (como "device" para o computador do usuário e "node" para servidores), os artefatos neles instalados e os protocolos e caminhos de comunicação entre esses nós, exatamente como no exemplo clássico de servidor de aplicação, servidor de banco de dados e navegador cliente já estudado na apostila da disciplina. A asserção II é falsa por dois motivos: primeiro, o diagrama de implantação é, sim, essencialmente estático ele documenta uma topologia de distribuição que normalmente não muda a cada execução, e não uma sequência temporal de eventos; segundo, é o diagrama de componentes que descreve a estrutura estática e não a dependência do ambiente de execução, mas nenhum dos dois diagramas nem o de implantação, nem o de componentes descreve a ordem cronológica de mensagens entre processos: essa é justamente a função do diagrama de sequência. A questão explora a confusão entre diagramas estruturais (classes, componentes, implantação) e diagramas comportamentais/de interação (sequência, atividades, casos de uso), que possuem propósitos de modelagem distintos dentro da UML.

Questão 14 - Gabarito: D

Justificativa Geral: Todas as quatro afirmações apresentadas estão **corretas** segundo a especificação padrão da UML (*Unified Modeling Language*) para o **Diagrama de Componentes**:

Afirmação I (Correta): A notação oficial para uma **interface fornecida** (*provided interface*) é o símbolo de uma esfera ("bolinha" ou *lollipop*), ligada ao componente por uma linha contínua. Ela sinaliza publicamente as operações ou serviços que o componente disponibiliza para o restante do sistema.

Afirmação II (Correta): A notação para uma **interface requerida** (*required interface*) é o símbolo de um arco ou semicírculo aberto ("encaixe" ou *socket*). Ela expressa uma dependência explícita, indicando que o componente necessita desses serviços para operar.

Afirmação III (Correta): Estabelece corretamente a diferença de nível de abstração entre os dois diagramas: enquanto o **Diagrama de Classes** atua no nível lógico (classes, métodos, atributos e tipos do código-fonte), o **Diagrama de Componentes** atua no nível de arquitetura física/modular (módulos, *containers*, bibliotecas, microsserviços ou arquivos executáveis) e na forma como eles se conectam por meio de contratos (interfaces).

Afirmação IV (Correta): A junção visual da "bolinha" (interface fornecida) com o "encaixe" (interface requerida) forma a representação gráfica conhecida como **conector de montagem** (*assembly connector* ou "plugue e tomada"). Esse acoplamento visual simboliza que a dependência de um componente é inteiramente suprida pelos serviços expostos pelo outro.

Questão 15 – Gabarito: A

Justificativa Geral: A composição é uma forma estrita de agregação (parte-todo forte). Um ItemPedido não possui existência própria fora de um Pedido específico. Se o Pedido for excluído, todos os seus itens associados devem ser automaticamente eliminados. Portanto, a Asserção I é verdadeira, a Asserção II é verdadeira e justifica perfeitamente a relação de causa-efeito descrita na Asserção I.

Questão 16 - Discursiva

Gabarito: Por se tratar de questão discursiva, espera-se um diagrama de classes com a seguinte estrutura mínima:

• Classes: Usuario, Emprestimo, ItemAcervo (classe abstrata), LivroFisico, Ebook e AudioLivro.

• Herança: LivroFisico, Ebook e AudioLivro herdam de ItemAcervo, que deve ser marcada com a restrição {abstract}, pois não existe instância genérica de "item do acervo" sem que seja um de seus três tipos concretos.

• Associações: Usuario 1 —— 0..* Emprestimo (um usuário pode ter zero ou vários empréstimos; cada empréstimo pertence a exatamente um usuário); Emprestimo 0..* —— 1 ItemAcervo (cada empréstimo refere-se a exatamente um item; cada item pode estar associado a vários empréstimos ao longo do tempo, em momentos distintos).

• Atributos: ItemAcervo (codigo, titulo, autor, quantidadeEmprestimos); Emprestimo (dataRetirada, dataPrevistaDevolucao); Ebook não possui atributo adicional obrigatório pelo enunciado, mas pode registrar quantidadeReservas; AudioLivro (quantidadeDispositivosSincronizados).

• Polimorfismo: o método abstrato calcularPrazoDevolucao() deve ser declarado em ItemAcervo e sobrescrito em cada subclasse: LivroFisico → retorna dataRetirada + 14; Ebook → retorna dataRetirada + 7; AudioLivro → retorna dataRetirada + 21 + (2 × quantidadeDispositivosSincronizados).

• O método calcularDataDevolucao() de Emprestimo deve, em sua implementação, delegar a operação ao objeto ItemAcervo associado, evidenciando o uso do polimorfismo em tempo de execução (binding dinâmico).

